

M2 — ControlRoom + Encounter + Student dataclasses

Phase: 1 — Data **Status:** DONE (2026-05-26) **Blocked by:** M0, M1 **Blocks:** M3, M5, M8, M11 **Estimated effort:** 1.5 day · **Actual:** 1 day

1. Purpose

Introduce the in-memory abstractions that mirror the M1 schema: `ControlRoom` (the unit of instructor governance), `Student` (a learner registered to a room), and `Encounter` (a `ControlRoom` member; realized as the existing v6 `ControlSession` class extended in place with the new v7 fields). Move the module-level singleton from a single `ControlSession` to a `ControlRoom` that may contain N encounters. Preserve every v6 import path and call site through a compatibility shim — so single-patient mode is byte-for-byte identical to v6.

This is the load-bearing module: every later phase (routes, dashboard, scenes, debrief) reads these dataclasses.

2. Structure

Files:

- `portal/control_room.py` — **new**. Home of `ControlRoom`,

`Student`, the `Encounter` alias, the `_active_room` singleton, and the v6-compatible helper functions `get_active`, `get_by_join_code`, `end_active_room`, `create_room`, `get_active_room`, `list_encounters`, `_reset_for_tests`.

- `portal/control_session.py` — **modified**. `ControlSession` now

carries six new v7-only fields (`room_id`, `encounter_label`, `activity_id`, `chart_mode`, `patient_persona_id`, `assigned_student_ids`) defaulting to safe single-patient values. Module-level singleton `_active` removed; `create_session`, `get_active`, `get_by_join_code`, `end_active`, `set_state` now delegate to `control_room`.

Key dataclasses:

Class	Owner	Role
<code>ControlRoom</code>	<code>control_room.py</code>	A roomful of Encounters under one instructor. Holds encounters dict, students dict, status, optional caps.
<code>Student</code>	<code>control_room.py</code>	A learner registered to a room. Persisted in student table via M8.
<code>Encounter</code>	<code>control_room.py</code> (= <code>ControlSession</code>)	Alias for <code>ControlSession</code> . The v6 class promoted in place with v7 fields.
<code>ControlSession</code>	<code>control_session.py</code>	The v6 dataclass extended with <code>room_id</code> , <code>encounter_label</code> , <code>activity_id</code> , <code>chart_mode</code> , <code>patient_persona_id</code> , <code>assigned_student_ids</code> .

3. Uses

v6 callers (unchanged code paths):

- `portal/server.py` calls `control_session.get_active()` / `control_session.get_by_join_code()` from every chat / EHR / device route. These now resolve to `control_room` under the hood — no call-site change needed.

- The wizard's `create_session(...)` call at `finalize` creates a `ControlRoom-of-1` transparently.

v7 new callers (later modules):

- M3 (route refactor): switches mutator routes to address encounters by id rather than singleton.
- M4 (new room API): `POST /api/room/start` calls `control_room.create_room(...)` + multiple `add_encounter(...)`.
- M5 (dashboard): `GET /api/room/state` enumerates `_active_room.encounters` and `.students`.
- M8 (roster persistence): writes `Student` rows to DB; reads back on server start.

4. Functions (exported API surface)

`portal/control_room.py`

Symbol	Signature	Purpose	
<code>ControlRoom</code>	<code>dataclass(room_id, room_code, label, status, created_at, ended_at, haiku_rate_cap, voice_char_cap, encounters, students)</code>	The room aggregate.	
<code>Student</code>	<code>dataclass(student_id, display_name, room_id, assigned_encounter_id, registered_at, last_seen)</code>	A learner.	
<code>Encounter</code>	<code>= ControlSession</code>	Alias for v7 contexts.	
<code>create_room(label="")</code>	<code>-> ControlRoom</code>	Create + set active.	
<code>get_active_room()</code>	<code>`-> ControlRoom \</code>	None`	Read singleton.
<code>end_active_room()</code>	<code>-> None</code>	End every encounter, clear singleton.	
<code>get_active()</code>	<code>`-> Encounter \</code>	None`	v6-compatible — returns single encounter; raises if <code>N > 1</code> .

Symbol	Signature	Purpose	
<code>get_by_join_code(code)</code>	<code>`-> Encounter \</code>	None`	Search every encounter by join code.
<code>list_encounters()</code>	<code>-> Iterable[Encounter]</code>	All encounters in the active room.	
<code>ControlRoom.add_encounter(enc)</code>	method	Attach an Encounter, stamp its room_id.	
<code>ControlRoom.get_encounter_by_join_code(code)</code>	method	Per-room dispatch.	
<code>ControlRoom.freeze_all()</code>	method	Set every encounter to paused, room to frozen.	
<code>ControlRoom.resume_all()</code>	method	Inverse of <code>freeze_all</code> .	
<code>ControlRoom.end()</code>	method	End every encounter, set room to ended.	
<code>ControlRoom.add_student(name, ...)</code>	method	Roster a learner; returns Student.	
<code>ControlRoom.assign_student(sid, eid)</code>	method	Bind a student to an encounter.	
<code>_reset_for_tests()</code>	helper	Clear <code>_active_room</code> for test isolation.	

portal/control_session.py (now thin shim)

Symbol	Behavior
<code>ControlSession</code>	Same class, +6 v7 fields.
<code>Station, EhrStation, DeviceStation, TranscriptEntry</code>	Unchanged.
<code>create_session(...)</code>	Now creates / reuses a room-of-1 and adds the encounter.
<code>get_active()</code>	Delegates to <code>control_room.get_active</code> .
<code>get_by_join_code(code)</code>	Delegates to <code>control_room.get_by_join_code</code> .
<code>end_active()</code>	Delegates to <code>control_room.end_active_room</code> .
<code>set_state(state)</code>	Sets state on every encounter in the active room.

5. Limitations

- The room and its students are **in-memory only** at this milestone.

Persistence of Student rows lands in M8; persistence of room metadata is implicit (room is rebuilt from the wizard each session).

- `Encounter = ControlSession` is an alias, not a subclass. Type

checkers see them as the same class. If a future module needs to distinguish "this is an Encounter, not a legacy ControlSession," introduce a distinguishing field or subclass — do not rely on `isinstance`.

- `get_active()` raises when the room holds multiple encounters. This

is intentional: any v6 code path that should NOT silently pick "the first encounter" instead fails loud. M3 audits and updates the call sites that need to be encounter-id-aware.

- `set_state(state)` is a fan-out; it sets EVERY encounter's state.

Per-encounter state changes go through addressing by id in M3/M4.

- The `_active_room` singleton is not thread-safe for concurrent

writes (consistent with v6's `_active`). The portal is single-instructor, so this is fine; M16's WebSocket transport adds the ordering guarantee where it matters.

6. Test status

Test file	Asserts	Status	Last run
tests/v7/test_room_create_with_2_encounters.py	Room holds 2 distinct encounters with distinct join codes; both know their <code>room_id</code> ; reverse lookup by id works; <code>room_code</code> distinct from join codes.	PASS	2026-05-26
tests/v7/test_get_active_returns_only_encounter_in_single_mode.py	<code>create_session</code> makes a room-of-1; <code>get_active</code> returns the single encounter; <code>get_active</code> raises on a multi-encounter room; <code>end_active</code> clears the singleton.	PASS (3 cases)	2026-05-26
tests/v7/test_get_by_join_code_finds_across_encounters.py	Per-encounter dispatch by case-insensitive join code; unknown code returns None; per-encounter station state does not bleed between encounters.	PASS (2 cases)	2026-05-26

7. Change list

Date	Author	Change	Files
2026-05-26	claude-codé	Added ControlRoom, Student, Encounter alias, singleton + helpers. Extended ControlSession with v7 fields. Re-pointed control_session v6-compat helpers to delegate to control_room. 6 acceptance tests across 3 test files.	portal/control_room.py (new), portal/control_session.py, tests/v7/test_room_create_with_2_encounters.py, tests/v7/test_get_active_returns_only_encounter_in_single_mode.py, tests/v7/test_get_by_join_code_finds_across_encounters.py

8. Open questions / known issues

- `create_session(...)` currently ends the prior active room if one exists (to match v6's "one active session at a time" behavior in single-patient mode). When M6 lands the wizard's Room-of-N branch, that branch must NOT end the prior room — it builds its own room from `control_room.create_room()` directly. The diverging behavior is the right one but the boundary deserves a test in M6.
- `Encounter.assigned_student_ids` is a plain list, not a set, to preserve insertion order. Insert-deduplication is handled by the `assign_student` method on `ControlRoom`. If a future module bypasses that method (e.g. restores from DB), it must dedupe.
- The Student `last_seen` field is currently `None` at construction — the `touch()` helper sets it. Decide M8's convention for initialization (likely set `last_seen = registered_at` on create).